Java Spring

Dr. Öğr. Üyesi Alper Talha KARADENİZ

Spring Nedir?

- Spring framework (kısaca Spring), Java ekosisteminin bir parçası olan ve kurumsal Java uygulamaları geliştirmeyi kolaylaştırmak amacıyla tasarlanmış kapsamlı bir uygulama çerçevesidir (application framework).
- Temel olarak, bir uygulamanın temel yapısını (iskeletini) sağlayarak programcıların uygulamayı sıfırdan yazma zahmetini ortadan kaldırır.

Spring Nedir?

- Spring, basit bir kütüphaneden ziyade devasa bir ekosistemdir. En temel düzeyde Spring, uygulama bileşenlerini (bunlara bean denir) oluşturan ve yöneten bir konteynere (Spring application context) sahiptir.
- Bu bileşenler, Bağımlılık Enjeksiyonu (Dependency Injection - DI) adı verilen bir yöntemle birbirine bağlanarak eksiksiz bir uygulama oluşturur.

Spring Ekosistemi

- **Spring Core:** Uygulama bağlamı ve bağımlılık enjeksiyonu gibi temel özellikleri sağlayan temel yapıdır.
- **Spring Boot:** "Yapılandırma yerine kurallar" (convention-over-configuration) yaklaşımını kullanarak uygulama kurulumunu ve bağımlılık yönetimini büyük ölçüde basitleştiren bir projedir.
- **Spring MVC:** Web uygulamaları ve REST servisleri geliştirmek için kullanılan web çerçevesidir.
- **Spring Data:** Veritabanlarına (SQL ve NoSQL) erişimi ve veri kalıcılığını kolaylaştıran bir katmandır.
- **Spring Security:** Uygulamanın kimlik doğrulama ve yetkilendirme gibi güvenlik ihtiyaçlarını karşılar.

Spring Neden Var?

- **Geliştirmeyi Kolaylaştırmak:** Spring, başlangıçta EJB (Enterprise JavaBeans) gibi ağır teknolojilere hafif bir alternatif olarak ortaya çıkmıştır ve temel misyonu Java uygulama geliştirmeyi kolaylaştırmaktır.
- **Tekrarlayan Kodları (Boilerplate) Ortadan Kaldırmak:** Birçok uygulama; günlükleme (logging), işlemler (transactions) ve güvenlik gibi benzer altyapı ihtiyaçlarına sahiptir. Spring, bu "tesisat" (plumbing) işlerini otomatikleştirerek geliştiricinin doğrudan iş mantığına (business logic) odaklanmasını sağlar.

Spring Neden Var?

- **Gevşek Bağlılık (Loose Coupling):** Kontrolün Tersine Çevrilmesi (Inversion of Control - IoC) prensibi sayesinde, nesnelerin oluşturulması ve yönetilmesi uygulamanın kendisinden alınarak framework'e devredilir. Bu, kodun daha esnek, bakımı kolay ve test edilebilir olmasını sağlar.
- **Modern İhtiyaçlara Uyum:** Günümüzde sadece tarayıcı tabanlı web uygulamaları değil; bulut tabanlı mikroservisler, reaktif programlama ve Kubernetes entegrasyonu gibi modern yapılar ön plandadır. Spring, bu yeni nesil geliştirme modellerine büyük üretkenlik artışı sağlar.
- **Güvenilirlik ve Topluluk Desteği:** Spring, Java geliştiricilerinin yarısından fazlası tarafından kullanılmaktadır. Bu devasa topluluk, karşılaşılan sorunların daha hızlı çözülmesini ve framework'ün sürekli test edilerek hatalardan arındırılmasını sağlar.

IoC (Inversion of Control)

- IoC (Inversion of Control - Kontrolün Tersine Çevrilmesi), bir uygulamanın yürütme kontrolünün uygulama kodunun kendisinden alınıp başka bir yazılım parçasına (Spring örneğinde framework'e) devredilmesi prensibidir.
- **Tersine Çevrilmiş Kontrol:** Geleneksel programlamada uygulama kendi akışını kontrol eder ve ihtiyaç duyduğu bağımlılıkları kendisi oluşturur. IoC modelinde ise bu kontrol framework'e geçer; yani framework uygulamayı ve onun kodunu yönetir.

IoC (Inversion of Control)

- **Yönetilen Eylemler:** Burada bahsedilen "kontrol", nesne örneklerinin (instance) oluşturulması veya belirli metotların çağrılması gibi işlemleri kapsar.
- **Konteyner Yapısı:** Spring, bu kontrolü sağlamak için Spring Application Context adı verilen bir konteyner sunar. Bu konteyner, uygulama bileşenlerini (bean'leri) oluşturur ve yönetir.
- **Dependency Injection (DI) İlişkisi:** Bağımlılık Enjeksiyonu, IoC prensibinin bir uygulamasıdır. Bileşenlerin kendi bağımlılıklarını oluşturup yaşam döngülerini yönetmeleri yerine, konteyner tüm bileşenleri oluşturur ve onları ihtiyaç duyan diğer bileşenlere enjekte eder.

Dependency Injection (DI)

- Dependency Injection (DI) (Bağımlılık Enjeksiyonu), bileşenlerin (bean'lerin) birbirine bağlanmasını temel alan bir tasarım kalıbıdır., Bu yöntemde, bir uygulamanın parçaları ihtiyaç duydukları bağımlılıkları kendileri oluşturup yaşam döngülerini yönetmek yerine, bu işi harici bir yapıya (Spring'deki konteyner) bırakırlar.

DI’nin Temel Özellikleri

- **Kontrolün Devri:** Bağımlılık enjeksiyonu kullanan bir uygulamada, nesneleri oluşturma ve ihtiyaç duyan sınıflara yerleştirme görevi Spring uygulama bağlamına (application context) aittir. Konteyner, bileşenleri oluşturur ve bunları ilgili sınıflara "enjekte" eder.
- **IoC ile İlişkisi:** DI, Inversion of Control (IoC) prensibinin pratik bir uygulamasıdır. Geleneksel programlamada uygulama kendi akışını kontrol ederken, DI modelinde kontrol framework'e geçer; framework, metod çağrılarını ve nesne yönetimini üstlenir.

DI’nin Temel Özellikleri

- **Esneklik ve Test Edilebilirlik:** Bileşenlerin bağımlılıklarından ayrıştırılması (decoupling), kodun daha esnek, bakımı kolay ve test edilebilir olmasını sağlar. Örneğin, bir birim testinde (unit test) gerçek bir nesne yerine kontrol edilebilir "sahte" bir nesne (mock) enjekte edilerek izole testler yapılabilir.

Spring'de DI Uygulama Yöntemleri

- **Yapıcı Metot (Constructor) Enjeksiyonu:** Bağımlılıkların sınıfın yapıcı metodu aracılığıyla sağlandığı yöntemdir; üretim kodunda en çok önerilen yaklaşımdır.
- **Setter Metot Enjeksiyonu:** Bağımlılıkların setter metotları üzerinden aktarıldığı yöntemdir.
- **Alan (Field) Enjeksiyonu:** Genellikle @Autowired anotasyonu kullanılarak bağımlılığın doğrudan sınıfın içine enjekte edilmesidir.

@SpringBootApplication ne yapar?

- **@SpringBootConfiguration:** Bu anotasyon, sınıfın bir yapılandırma (configuration) sınıfı olduğunu belirtir. Aslında @Configuration anotasyonunun özelleşmiş bir formudur ve Spring uygulama bağlamına (application context) bean'ler sağlamanıza olanak tanır.
- **@EnableAutoConfiguration:** Spring Boot'un meşhur otomatik yapılandırma (autoconfiguration) özelliğini etkinleştirir. Sınıf yolundaki (classpath) kütüphanelere, ortam değişkenlerine ve diğer faktörlere bakarak uygulamanın ihtiyaç duyabileceği bileşenleri tahmin eder ve bunları sizin yerinize otomatik olarak yapılandırır.

@SpringBootApplication ne yapar?

- **@ComponentScan:** Bileşen taramayı etkinleştirir. Bu sayede Spring; @Component, @Controller, @Service ve @Repository gibi anotasyonlarla işaretlenmiş sınıfları otomatik olarak keşfeder ve bunları Spring uygulama bağlamında birer bileşen (bean) olarak kaydeder.

@Component nedir?

- En temel ve genel amaçlı stereotype anotasyonudur. Bir sınıfı bu anotasyonla işaretlediğinizde, Spring'in bileşen tarama (component scanning) mekanizması bu sınıfı otomatik olarak keşfeder, bir örneğini oluşturur ve bu örneği Spring uygulama bağlamına (application context) bir bean olarak kaydeder.

@Service nedir?

- @Component anotasyonunun özelleşmiş bir formudur ve sınıfa bir "servis" rolü yükler. Genellikle iş mantığının (business logic) ve kullanım durumlarının (use cases) uygulandığı sınıfları işaretlemek için kullanılır. Teknik olarak @Component ile aynı şekilde çalışsa da, sınıfın amacını daha net bir şekilde belirtir.

@Repository nedir?

- Veri erişim katmanındaki sınıfları işaretlemek için kullanılır ve bir "depo" (repository) sorumluluğunu temsil eder. Bu anotasyon, sınıfın veritabanı işlemlerinden (veri kalıcılığı/persistence) sorumlu olduğunu belirtir. @Component gibi çalışmasının yanı sıra, veri erişim katmanına özgü ek yetenekler (hata yakalama mekanizmaları gibi) sunabilir.

@Autowired nedir?

- @Autowired, Spring Framework'te Bağımlılık Enjeksiyonu (Dependency Injection - DI) işlemini otomatikleştirmek için kullanılan temel anotasyondur,. Spring'e, ihtiyaç duyulan bir bileşeni (bean) uygulama bağlamından (context) bulup ilgili yere yerleştirmesi talimatını verir.
- Yapıcı Metot (Constructor) Enjeksiyonu vurgusuyla @Autowired kullanımı, modern Spring geliştirmede en çok önerilen ve kullanılan yaklaşımdır. Bu yöntemde @Autowired anotasyonu sınıfın yapıcı metodunun üzerine eklenir ve Spring, bu metodun parametreleri için gerekli olan bean'leri otomatik olarak sağlar.